

Analysis of Required Link Bandwidth for Wireless Full-Mesh Networks

Woo-Nyeong Choe, Tae-In Oh, and Young-Chai Ko

School of Electrical Engineering
Korea University
Seoul, South Korea
{hyy741, logan10, koyc}@korea.ac.kr

June 2026

Abstract

This thesis analyzes the minimum per-link wireless bandwidth required for a wireless full-mesh network to match representative wired high-performance computing (HPC) datacenter network topologies. The wired baselines are Fat-tree, Jellyfish, Dragonfly, Torus, and Hypercube, while the wireless baseline is an idealized full-mesh over N compute endpoints. The comparison is formulated as a required-minimum-bandwidth problem: for a wired per-link bandwidth B_{wired} , what minimum value of B_{wireless} is required for the wireless design to match the wired baseline under the same metric?

The thesis separates structural and traffic-aware metrics. The main analytical result uses bisection width and bisection bandwidth because they give closed-form required minimum wireless bandwidth expressions for the selected topology families. A secondary analysis uses the maximum-concurrent-flow throughput framework of Jyothi et al. to check how the required minimum changes under traffic matrices such as random permutation, all-to-all, and longest matching [6]. This separation is necessary because cut metrics can be useful first-order indicators but do not fully determine traffic-dependent throughput.

The result is that the answer depends strongly on the wireless model. Under the ideal full-mesh bisection model, an equal cut has $N^2/4$ possible cross-cut wireless links, so the required per-link wireless bandwidth decreases as $O(1/N)$. Under a connection-limited model where each endpoint can maintain only c active wireless links at a time, the same bisection requirement changes to $O(1/c)$. Therefore, the practical bottleneck is not only per-link bandwidth; it is also the number of simultaneous active wireless links each endpoint can sustain.

Keywords: HPC network topology; wireless full-mesh; bisection bandwidth; maximum concurrent flow; TopoBench; Fat-tree; Jellyfish; Dragonfly; Torus; Hypercube

1 Introduction

Network topology is a central design factor in HPC and datacenter systems because it affects communication capacity, path length, port usage, and deployment complexity. Fat-tree, Jellyfish, Dragonfly, Torus, and Hypercube represent different tradeoffs among hierarchy, randomness, locality, low diameter, and port cost [1, 2, 3, 4, 5].

Existing wired topologies require many switches, cables, and switch ports to improve performance and scalability. Multi-hop routing increases path length and creates link contention on intermediate links, while cabling complexity and deployment cost grow as the network scales. If

sufficiently capable wireless links become available, endpoints could communicate through one-hop direct connections, reducing the need for complex cabling and intermediate switch-to-switch links. A wireless full-mesh topology extends this idea to every endpoint pair, provides maximum direct connectivity, and allows the flexibility of dynamic link reconfiguration. For simplicity, this study uses an idealized wireless full-mesh as the wireless baseline.

This thesis does not propose a new wired topology. Instead, it asks a required-minimum-bandwidth question: how large must the per-link bandwidth of a wireless full-mesh network be to match or outperform each wired baseline? The core research question is:

What minimum per-link wireless bandwidth B_{wireless} is required for a wireless full-mesh network to match or outperform wired HPC topologies?

A wireless full-mesh network has direct connectivity between every pair of compute endpoints. For N endpoints, the total number of direct links is

$$E_{\text{total}} = \frac{N(N-1)}{2}, \quad (1)$$

the diameter is one hop, and an equal bisection has

$$BW_{\text{wireless,ideal}} = \left\lfloor \frac{N}{2} \right\rfloor \left\lceil \frac{N}{2} \right\rceil \approx \frac{N^2}{4} \quad (2)$$

possible cross-cut links for even N . These properties make the ideal full-mesh model look extremely strong under bisection bandwidth, but they also expose the model’s feasibility problem: realizing every direct pair or allowing every node to maintain $N-1$ simultaneous active peers is not realistic for large N .

For this reason, the thesis uses three comparison models. The first is an ideal bisection model where every wireless pair can be active simultaneously. The second is a connection-limited bisection model where each node can maintain at most c active wireless links. The third is a TopoBench-style effective-throughput model that compares traffic-specific throughput scaling factors. The main contribution is a consistent required-minimum-bandwidth framework that keeps these models separate. The thesis shows why the optimistic $O(1/N)$ result from ideal bisection should be treated as an upper bound, and why the more practical connection-limited model makes the active-link budget c the dominant feasibility parameter.

2 Background and Related Work

2.1 Wired topology baselines

Fat-tree is a Clos-style hierarchical datacenter topology designed to provide high aggregate bandwidth using commodity switches. In the common k -ary Fat-tree construction, the topology uses edge, aggregation, and core switching layers and supports $k^3/4$ hosts [1].

Jellyfish is a random regular graph topology proposed for datacenters. It connects switches randomly while reserving some switch ports for servers, and it was introduced as a flexible topology that can achieve high throughput with the same equipment budget as more structured topologies [2].

Dragonfly is a group-based topology that uses local links inside router groups and global links between groups to build large low-diameter systems. The Dragonfly family is included as a representative modern HPC topology family, but the exact numerical results depend on parameterization [3].

Torus networks connect nodes through nearest-neighbor grid-like links and are a classical locality-oriented HPC interconnect family. A uniform even d -dimensional torus has a bisection expression that can be written analytically in terms of N and d [4].

Hypercube networks use a recursive dimension-based structure. An m -dimensional hypercube has $N = 2^m$ nodes, degree m , and bisection width $2^{m-1} = N/2$ [5].

Table 1: Wired topology baselines: structure and reason for inclusion.

Topology	Structure	Why included
Fat-tree	Hierarchical Clos-style topology	Common high-bisection datacenter baseline
Jellyfish	Random regular graph	High-throughput random-graph baseline
Dragonfly	Local groups plus global links	Representative low-diameter HPC topology
Hypercube	Dimension-based recursive graph	Theoretical low-diameter baseline
Torus	Nearest-neighbor grid	Classical HPC locality baseline

2.2 Traffic-aware throughput and TopoBench

Bisection bandwidth is a structural metric: it measures the minimum capacity crossing an equal cut. It is useful because it is often analytically tractable, but it does not specify a traffic matrix or routing solution.

Jyothi et al. define topology throughput for a network G and traffic matrix T as the maximum scaling factor t such that tT can be routed while satisfying link-capacity and flow-conservation constraints [6]. This can be written as

$$t^*(G, T) = \max \{t : tT \text{ is feasible in } G\}. \quad (3)$$

This is a maximum-concurrent-flow problem. The TopoBench artifact associated with this framework generates linear programs for selected topologies and traffic modes.

It is important to distinguish the traffic matrix T from the implementation-level objective variable K . T denotes the offered traffic demand matrix, while K is the scaling objective printed by the local TopoBench-style linear program for the generated demand set. These are different objects. In particular, raw K need not be numerically equal to Jyothi et al.’s hose-normalized throughput t ; the distinction can appear in traffic modes such as all-to-all because the generated demand matrix and the paper-level normalization may not be identical. One concrete source of this divergence is that the TopoBench LP does not normalize server-side demands by the number of servers, whereas the paper’s throughput factor t is defined with each server’s demand normalized to unit total; this implementation difference means raw K values are not numerically equal to t even for the same topology and traffic mode. However, if two values K_1 and K_2 are computed for the same traffic mode and the same network size, then the unnormalized scaling convention is common to both runs. Their ratio K_1/K_2 is therefore meaningful as a relative throughput or required-bandwidth comparison. This thesis uses K -ratios only in this relative sense, not as a claim that raw artifact K is numerically identical to Jyothi et al.’s t .

Jyothi et al. also use relative throughput when comparing topology families. Their reason is that raw throughput does not by itself account for differences in equipment and scale. They therefore compare a topology against a random graph built with the same equipment and under the same traffic matrix, and define relative throughput by normalizing the topology throughput by that random-graph throughput [6]. This thesis follows the same spirit only as a trend-comparison

device: the reconstructed K values are intended to preserve relative direction under comparable traffic assumptions, not to claim exact numerical equivalence with the paper’s reported t values.

The TopoBench LP assumes server-to-server link capacity normalized to one, switch-to-switch link capacity equal to one, and server-to-switch link capacity treated as infinite [6]. Under this normalization, the scaling factor t has a direct interpretation: $t > 1$ means the topology can carry strictly more than the base traffic demand; $t = 1$ means it saturates exactly at the base demand; and $t < 1$ means the topology cannot fully support the base demand.

2.3 Metric classification

For the purpose of this thesis, I classify network topology metrics into four broad categories: capacity (how much traffic the network can carry), delay (path length and communication latency), cost (the number of switches, cables, and ports required to build and deploy), and robustness (connectivity and performance under node or link failures). This classification is used as an analysis framework for the minimum-bandwidth question, not as a claim that these are the only possible metric categories.

Capacity metrics divide into two types. *Traffic-independent* capacity evaluates the structural bandwidth potential of a topology from its graph alone, without specifying a demand matrix; bisection bandwidth is the canonical example and can often be derived analytically. *Traffic-dependent* capacity evaluates how much traffic the topology can carry under a specific traffic matrix; it requires defining traffic demands and solving a flow or routing problem. The maximum concurrent flow throughput of Jyothi et al. is the traffic-dependent capacity metric used in this thesis [6].

3 Metrics and Problem Definition

The notation follows the presentation slides: B is per-link bandwidth, BW is bisection width, and BB is bisection bandwidth. Thus $BB = BW \cdot B$. The main output is the relative wireless link bandwidth $B_{\text{wireless}}/B_{\text{wired}}$.

The structural required minimum wireless bandwidth relative to wired bandwidth is defined by asking when the wireless bisection bandwidth is at least the wired bisection bandwidth:

$$\frac{BB_{\text{wireless}}}{BB_{\text{wired}}} = \frac{BW_{\text{wireless}}B_{\text{wireless}}}{BW_{\text{wired}}B_{\text{wired}}} \geq 1, \quad (4)$$

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{BW_{\text{wired}}}{BW_{\text{wireless}}}. \quad (5)$$

The traffic-aware required minimum wireless bandwidth uses relative throughput. Jyothi et al. define throughput using a scaling factor t , while the TopoBench implementation reports an LP objective K for the generated demand set [6]. Under the same traffic mode and network size, this thesis uses the relative K -ratio as the estimate of the relative t -ratio:

$$\frac{\Theta_{\text{wireless}}}{\Theta_{\text{wired}}} = \frac{t_{\text{wireless}}B_{\text{wireless}}}{t_{\text{wired}}B_{\text{wired}}} \geq 1, \quad (6)$$

$$\frac{t_{\text{wireless}}}{t_{\text{wired}}} = \frac{K_{\text{wireless}}}{K_{\text{wired}}}, \quad (7)$$

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{K_{\text{wired}}}{K_{\text{wireless}}}. \quad (8)$$

Table 2: Notation used in the required-bandwidth analysis.

Symbol	Meaning
N	Number of compute endpoints or servers
N_{sw}	Number of Jellyfish switches
k	Switch radix or number of switch ports
r	Jellyfish network-facing ports per switch
d	Torus dimension
q	Torus side length
p, a, g	Dragonfly terminals per router, routers per group, and groups
m	Hypercube dimension
B_{wired}	Wired per-link bandwidth
B_{wireless}	Wireless per-link bandwidth
BW_{wired}	Wired bisection width, measured as links crossing a minimum equal cut
BB_{wired}	Wired bisection bandwidth, $BW_{\text{wired}}B_{\text{wired}}$
BB_{wireless}	Wireless bisection bandwidth, $BW_{\text{wireless}}B_{\text{wireless}}$
c	Maximum simultaneous active wireless links per endpoint
μ	Maximum requested wireless pairs incident to one endpoint under traffic matrix T
K_{wired}	Wired throughput scaling factor
K_{wireless}	Analytical wireless scaling factor for the same traffic mode
Θ	Traffic-aware throughput, represented as scaling factor times link bandwidth

4 Wireless Full-Mesh Models

4.1 Ideal full-mesh bisection

For even N , an equal cut divides the endpoints into two sets of size $N/2$. In an ideal full-mesh, every endpoint on one side can connect directly to every endpoint on the other side, giving $N^2/4$ possible cross-cut wireless links. The ideal wireless bisection width is

$$BW_{\text{wireless}} = \left\lfloor \frac{N}{2} \right\rfloor \left\lceil \frac{N}{2} \right\rceil \approx \frac{N^2}{4}. \quad (9)$$

Matching a wired bisection width BW_{wired} requires

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{BW_{\text{wired}}}{BW_{\text{wireless}}} \approx \frac{4BW_{\text{wired}}}{N^2}. \quad (10)$$

This model is intentionally optimistic. Its low required bandwidth comes from assuming that all cross-cut wireless pairs can be active independently at the same time. The total full-mesh direct-link count remains $N(N-1)/2$, so good bisection bandwidth does not imply practical deployability.

4.2 Connection-limited bisection

The connection-limited model introduces c , the maximum number of active wireless links that one endpoint can maintain simultaneously. In an equal cut, one side has $N/2$ endpoints, so that side can expose at most $cN/2$ cross-cut wireless endpoints. The number of possible cross-cut pairs is still approximately $N^2/4$. For the practical case $c \ll N$, the active cross-cut bisection width is approximately

$$BW_{\text{wireless}} \approx \frac{cN}{2}. \quad (11)$$

The corresponding required minimum wireless bandwidth related to wired bandwidth is

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{BW_{\text{wired}}}{BW_{\text{wireless}}} \approx \frac{2BW_{\text{wired}}}{cN}. \quad (12)$$

The key change is the scaling law. Ideal bisection gives a required bandwidth proportional to $1/N$, while connection-limited bisection gives a required bandwidth proportional to $1/c$. Thus, the active-link budget plays a role similar to radix or port count in wired networks.

4.3 Wireless scaling factor for traffic-aware comparison

The traffic-aware section uses an analytical wireless K_{wireless} , not a packet-level or physical-layer wireless simulation. This value is a required-bandwidth denominator derived from the same common-scaling idea used by TopoBench, but with the bottleneck constraint coming from the wireless active-link budget c .

If a bottleneck endpoint is incident to $\mu(T)$ requested wireless pairs under traffic matrix T , then scaling every requested pair by K requires $\mu(T)K$ active-link service units at that endpoint. Since the endpoint has budget c , the wireless constraint is

$$\mu(T)K \leq c, \quad (13)$$

$$K_{\text{wireless}}(T, c) = \frac{c}{\mu(T)}. \quad (14)$$

Table 3: Analytical wireless scaling factor by traffic mode.

Mode	Workload	$\mu(T)$	Wireless constraint	K_{wireless}
0	Random permutation	2	$2K_{\text{wireless}} \leq c$	$c/2$
1	All-to-all	$N - 1$	$(N - 1)K_{\text{wireless}} \leq c$	$c/(N - 1)$
4	Longest matching	1	$K_{\text{wireless}} \leq c$	c

Mode 0 and mode 4 have model caveats. Mode 0 uses $\mu = 2$ because source and destination participation are counted against the same endpoint active-link budget. Mode 4 uses $\mu = 1$ because reciprocal directed demands are interpreted as sharing one full-duplex active peer link. Different transmit/receive or half-duplex assumptions would change these denominators.

5 Wired Topology Baselines

The bisection-width inputs used for the structural comparison are summarized below. The relative BW column gives $BW_{\text{wired}}/BW_{\text{wireless}}$ under the ideal full-mesh bisection width $BW_{\text{wireless}} \approx N^2/4$.

Table 4: Bisection-width inputs and ideal relative BW for wired baselines.

Topology	Bisection-width input BW_{wired}	Relative $BW_{\text{wired}}/BW_{\text{wireless}}$
Fat-tree	$N/2$	$2/N$
Jellyfish	$Nr/(4(k - r))$ lower bound	$r/((k - r)N)$
Hypercube	$N/2$, for $N = 2^m$	$2/N$
Dragonfly	$\lfloor g/2 \rfloor \lceil g/2 \rceil$	$4 \lfloor g/2 \rfloor \lceil g/2 \rceil / N^2$
Torus, even dimensional	$2N^{1-1/d}$	$8/N^{1+1/d}$

For the numerical structural examples, the main server counts are $N = 1024$ for Fat-tree, Jellyfish, Hypercube, and the three-dimensional Torus example, and $N = 1056$ for the Dragonfly row.

For the Dragonfly row, the bisection expression starts from the product of the two sides of the group-level cut:

$$BW_{\text{wired,Dragonfly}} = \left\lfloor \frac{g}{2} \right\rfloor \left\lceil \frac{g}{2} \right\rceil = \frac{2N + \sqrt{1 + 4N} - 1}{8} \approx \frac{N + \sqrt{N}}{4}. \quad (15)$$

6 Methodology

The structural analysis first substitutes each wired topology’s bisection width into the ideal wireless required-bandwidth expression:

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{4BW_{\text{wired}}}{N^2}. \quad (16)$$

It then substitutes the same wired baseline into the connection-limited required-bandwidth expression:

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{2BW_{\text{wired}}}{cN}. \quad (17)$$

The traffic-aware analysis uses computed K_{wired} values under traffic modes 0, 1, and 4. For each traffic mode, the effective-throughput required bandwidth is computed as

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{K_{\text{wired}}}{K_{\text{wireless}}}. \quad (18)$$

The numerical traffic-aware table uses reconstructed K values derived from Jyothi et al.’s Table I relative-throughput trends and a local Jellyfish raw- K anchor. Direct local values are kept only in the appendix as a sensitivity check; local files and scripts are intentionally not listed as bibliographic references.

7 Results

7.1 Ideal Bisection: Required Minimum Wireless Bandwidth

The ideal bisection results are structural upper-bound results. They assume all $N^2/4$ cross-cut wireless links can be active simultaneously.

Table 5: Ideal full-mesh bisection required minimum $B_{\text{wireless}}/B_{\text{wired}}$ by topology.

Topology	Required minimum $B_{\text{wireless}}/B_{\text{wired}}$
Fat-tree	$2/N$
Jellyfish	$r/((k-r)N)$
Hypercube	$2/N$, for power-of-two N
Dragonfly	$4 \lfloor g/2 \rfloor \lceil g/2 \rceil / N^2 = (2N + \sqrt{1 + 4N} - 1)/(2N^2)$
Torus, even d -dimensional	$8/N^{1+1/d}$

Ideal bisection relative BW values at the numerical N used in this thesis.

Topology	N	Relative $BW_{\text{wired}}/BW_{\text{wireless}}$
Fat-tree	1024	0.001953
Jellyfish, $r/(k-r) = 4$	1024	0.003906
Hypercube	1024	0.001953
Dragonfly	1056	0.000976
Torus, $d = 3$	1024	0.000775

For $N = 1024$, the Fat-tree and Hypercube ideal required minimum bandwidth is

$$\frac{2}{1024} = 0.001953, \quad (19)$$

or about 0.195% of a wired link. This number is small because the ideal model counts

$$\frac{1024^2}{4} = 262,144 \quad (20)$$

simultaneous cross-cut links. For context, the total number of full-mesh links at $N = 1024$ is

$$\frac{N(N-1)}{2} = \frac{1024 \times 1023}{2} = 523,776, \quad (21)$$

so the ideal cross-cut count of 262,144 represents approximately half of all possible full-mesh links. Activating all 262,144 of these links simultaneously is the assumption that makes the ideal model physically unrealistic.

Figure 1 shows how the ideal required bandwidth decreases with N across all five topology families. All curves converge toward zero, with Jellyfish requiring the largest relative wireless bandwidth and 3D Torus the smallest.

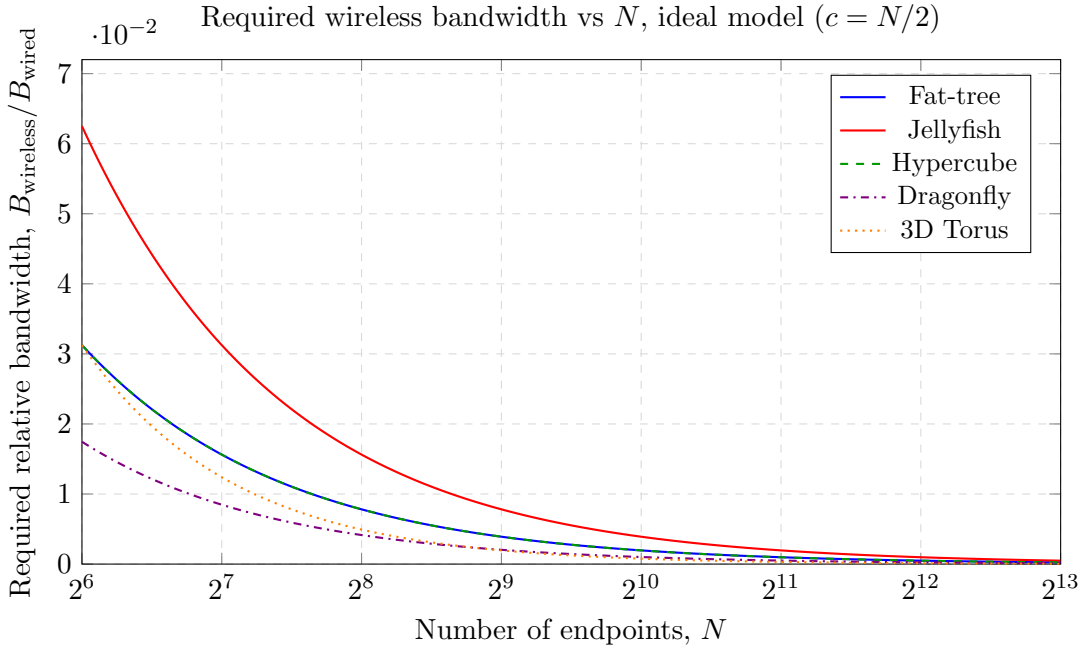


Figure 1: Ideal bisection required minimum relative bandwidth vs N . The ideal model assumes $c = N/2$ simultaneous cross-cut active links per side. Fat-tree and Hypercube overlap exactly at $2/N$. Jellyfish parameters: $k = 20$, $r = 16$, giving $r/(k-r) = 4$.

7.2 Connection-Limited Bisection: Required Minimum Wireless Bandwidth

With a per-node active-link limit c , the required bandwidth no longer decreases with N for Fat-tree and Hypercube. It depends directly on c .

The symbolic required minimum bandwidths follow directly from substituting each wired bisection width into the connection-limited expression $B_{\text{wireless}}/B_{\text{wired}} \geq 2BW_{\text{wired}}/(cN)$.

Table 6: Connection-limited bisection required minimum $B_{\text{wireless}}/B_{\text{wired}}$, symbolic form.

Topology	Required minimum $B_{\text{wireless}}/B_{\text{wired}}$
Fat-tree	$1/c$
Jellyfish	$r / (2(k - r)c)$
Hypercube	$1/c$, for power-of-two N
Dragonfly	$(2N + \sqrt{1 + 4N} - 1) / (4cN)$
Torus, even d -dimensional	$4 / (cN^{1/d})$

The Fat-tree and Hypercube symbolic results ($1/c$) are independent of N : once c is fixed, the required bandwidth does not improve as the system scales. Dragonfly converges to $1/(2c)$ for large N , while Torus decreases as $N^{-1/d}$ and therefore still improves with scale, but more slowly than the ideal model predicts.

Table 7: Connection-limited bisection required minimum $B_{\text{wireless}}/B_{\text{wired}}$, numerical values.

Topology	N	$c = 1$	$c = 2$	$c = 3$	$c = 4$
Fat-tree	1024	1.000000	0.500000	0.333333	0.250000
Jellyfish, $r/(k - r) = 4$	1024	2.000000	1.000000	0.666667	0.500000
Hypercube	1024	1.000000	0.500000	0.333333	0.250000
Dragonfly	1056	0.515152	0.257576	0.171717	0.128788
Torus, $d = 3$	1024	0.396850	0.198425	0.132283	0.099213

For the analytical Torus row, a uniform even d -dimensional torus has

$$\frac{B_{\text{wireless}}}{B_{\text{wired}}} \geq \frac{4}{cN^{1/d}}. \quad (22)$$

These results show the main structural conclusion. The ideal model suggests that very small wireless links can match wired bisection at large N , but the connection-limited model requires each wireless link to be a much larger fraction of a wired link unless c is large.

Figures 2–4 plot the required wireless bandwidth against N for representative topologies at $c = 1, 2, 3, 4$.

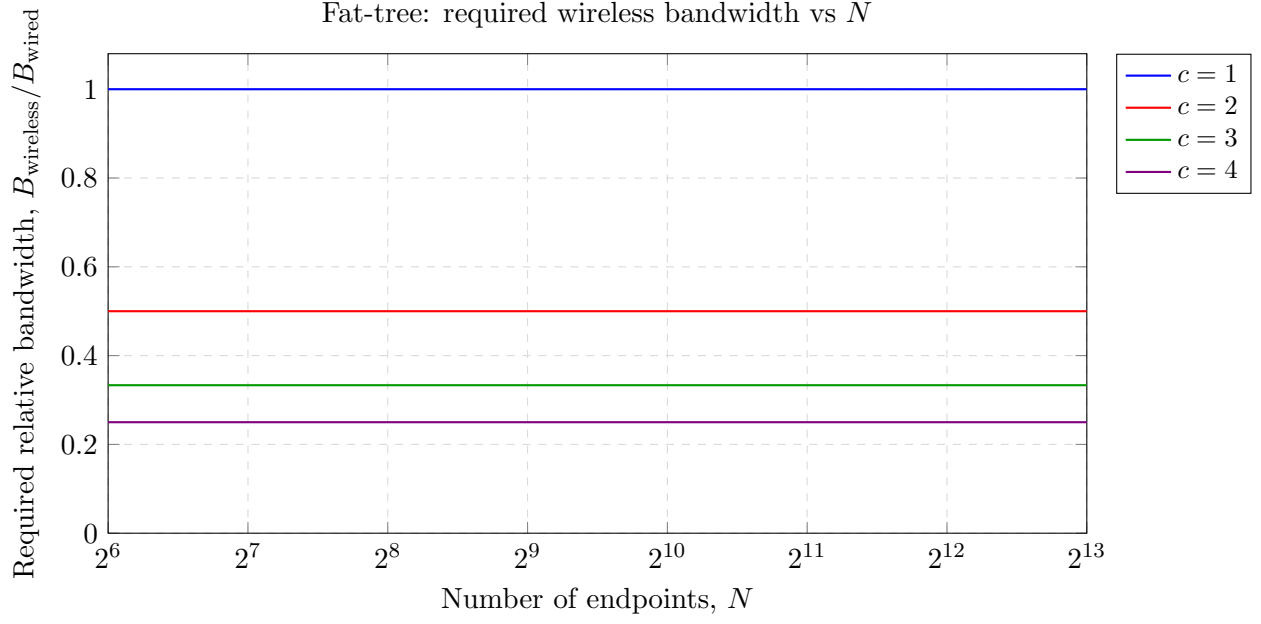


Figure 2: Fat-tree connection-limited required minimum relative bandwidth. The required bandwidth is $1/c$, independent of N , so it appears as a flat horizontal line for each value of c .

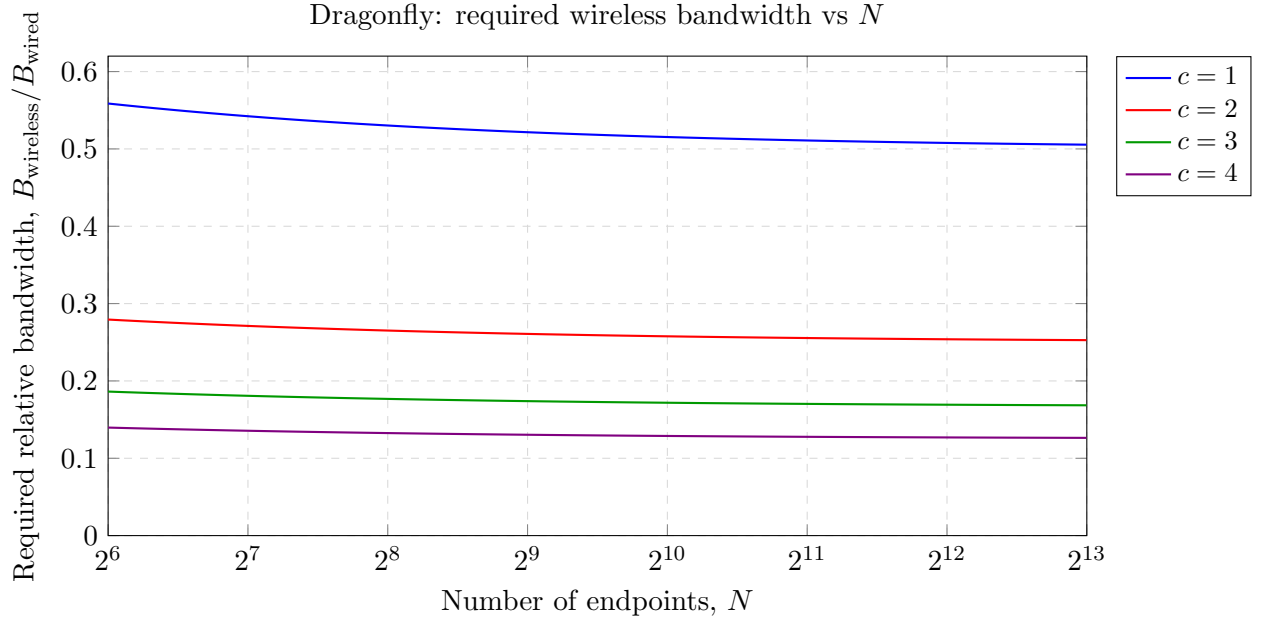


Figure 3: Dragonfly connection-limited required minimum relative bandwidth. The required bandwidth converges to $1/(2c)$ for large N due to the group-level cut structure.

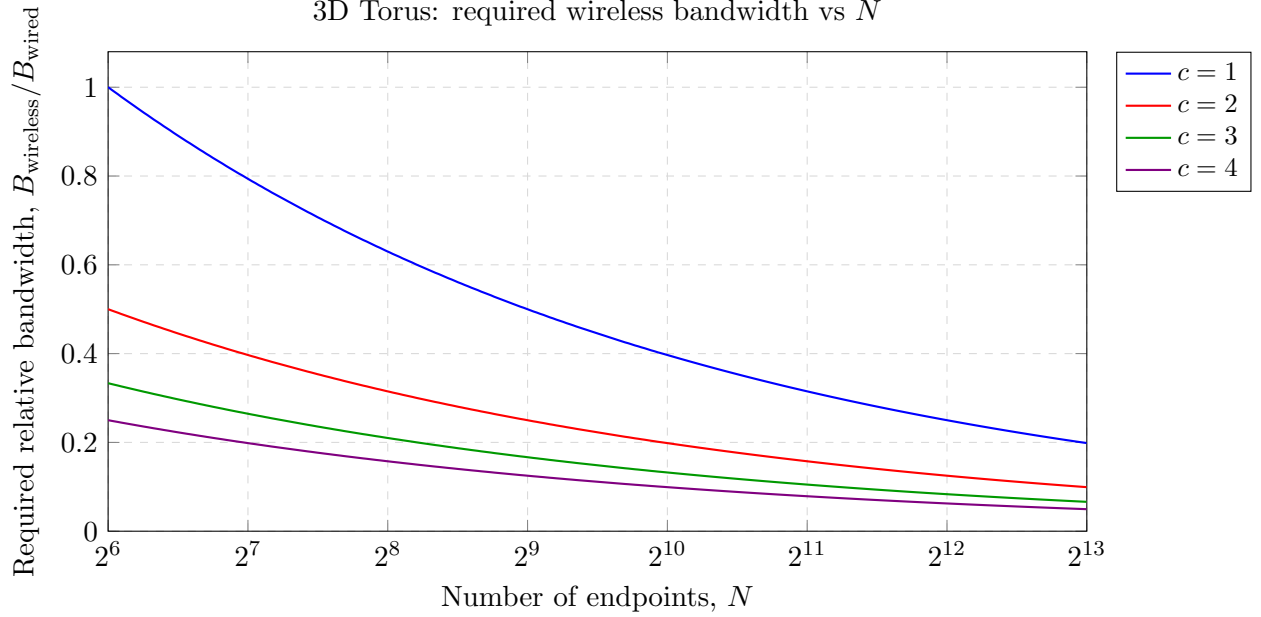


Figure 4: 3D Torus connection-limited required minimum relative bandwidth. The required bandwidth $4/(cN^{1/3})$ decreases with N , unlike Fat-tree and Hypercube, because the Torus bisection width grows faster than linearly in N .

7.3 Traffic-Aware Effective Throughput: Required Minimum Wireless Bandwidth

The traffic-aware analysis fixes $N = 1024$ servers. Jyothi et al. state that on a 32 GB machine the longest-matching LP scales to 1,024 servers, while the Kodialam TM can only reach 128 nodes before exhausting solver memory [6]. Therefore, $N = 1024$ is the practical LP-solver limit for the longest-matching workload, and it is used here as the representative scale for all three traffic modes.

Jyothi et al. Table I reports relative throughput trends, not raw TopoBench objective values [6]. Because the exact Jellyfish random-generation parameter needed to reproduce the SC 2016 table was not identified, this thesis uses a local $N = 1024$ Jellyfish run as the raw- K anchor. The local Fat-tree-to-Jellyfish ratios are close enough to the Table I relative direction to make the reconstruction feasible as an approximate trend check. The reconstructed estimate is

$$K_{\text{est}}(\text{topology}, T) = K_{\text{local}}(\text{Jellyfish}, T) \times t_{\text{Table I}}(\text{topology}, T). \quad (23)$$

The following table uses the reconstructed raw- K estimates also shown in the presentation. All entries are evaluated at $N = 1024$ servers. These values are not raw values printed by Jyothi et al.; they are reconstructed estimates from Table I relative throughput using the local Jellyfish anchor.

Table 8: Traffic-aware required minimum bandwidth using reconstructed K_{wired} estimates at $N = 1024$.

Topology	Workload		K_{wired}	K_{wireless}	Required $B_{\text{wireless}}/B_{\text{wired}}$	minimum
Jellyfish, local anchor	Random	permu-	1.38225161	$c/2$	$2.76450322/c$	
	tation					
Jellyfish, local anchor	All-to-all		0.00153672	$c/1023$	$1.57206456/c$	
Jellyfish, local anchor	Longest	match-	1.14560637	c	$1.14560637/c$	
	ing					
Fat-tree, recon-	Random	permu-	1.00904368	$c/2$	$2.01808736/c$	
structed	tation					
Fat-tree, recon-	All-to-all		0.00099887	$c/1023$	$1.02184401/c$	
structed						
Fat-tree, recon-	Longest	match-	1.01958967	c	$1.01958967/c$	
structed	ing					
Hypercube, recon-	Random	permu-	1.16109135	$c/2$	$2.32218270/c$	
structed	tation					
Hypercube, recon-	All-to-all		0.00110644	$c/1023$	$1.13188812/c$	
structed						
Hypercube, recon-	Longest	match-	0.58425925	c	$0.58425925/c$	
structed	ing					
Dragonfly, recon-	Random	permu-	1.05051122	$c/2$	$2.10102244/c$	
structed	tation					
Dragonfly, recon-	All-to-all		0.00145988	$c/1023$	$1.49345724/c$	
structed						
Dragonfly, recon-	Longest	match-	0.82483659	c	$0.82483659/c$	
structed	ing					

The traffic-aware required bandwidth values are much larger than the ideal bisection values because the wireless denominator is limited by endpoint active-link budget rather than by the number of possible full-mesh pairs. This comparison should be read as an approximate traffic-aware trend, not as a hardware-equivalent wireless simulation.

Figures 5–7 show the traffic-aware required bandwidth as a function of active-link budget c , with $N = 1024$ fixed. Each figure contains one line per wired topology. Since K_{wired} is constant and K_{wireless} is proportional to c , all curves follow a $1/c$ shape; the topology ordering and absolute level differ by traffic mode.

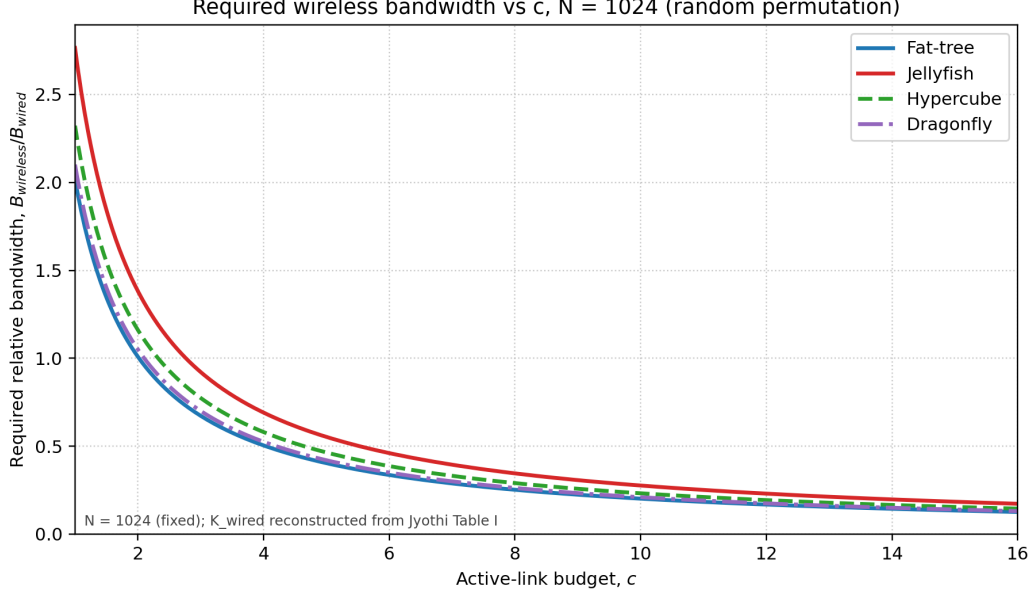


Figure 5: Mode 0 (random permutation): required minimum $B_{\text{wireless}}/B_{\text{wired}}$ vs active-link budget c at $N = 1024$. The required bandwidth is $2K_{\text{wired}}/c$. Jellyfish requires the highest wireless bandwidth; Fat-tree the lowest among the four topologies.

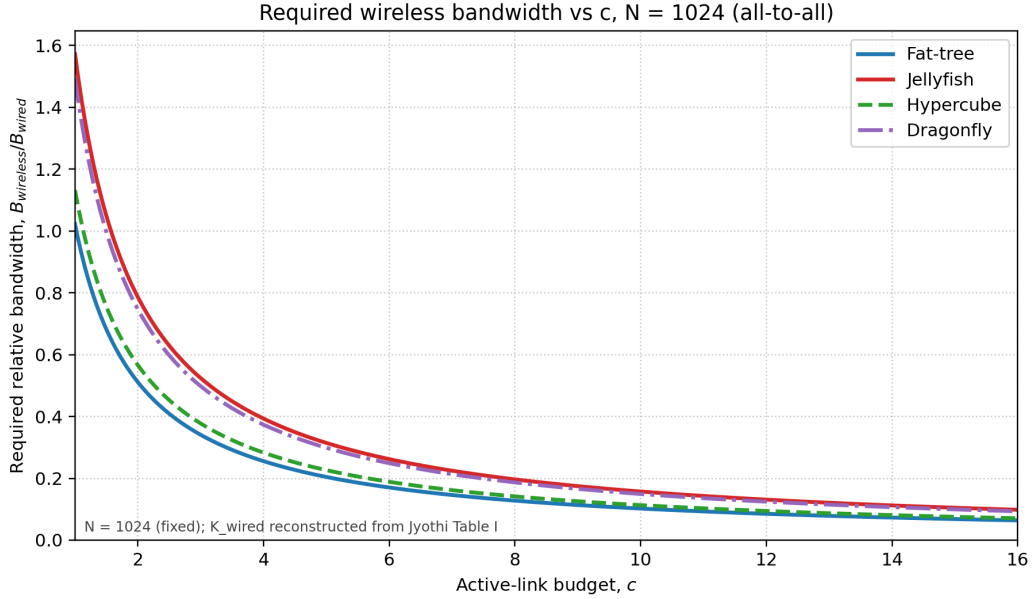


Figure 6: Mode 1 (all-to-all): required minimum $B_{\text{wireless}}/B_{\text{wired}}$ vs c at $N = 1024$. The required bandwidth is $K_{\text{wired}}(N - 1)/c$. At $N = 1024$, the absolute values are comparable to mode 0, but the all-to-all required bandwidth grows with N , so all-to-all becomes increasingly demanding at large scale.

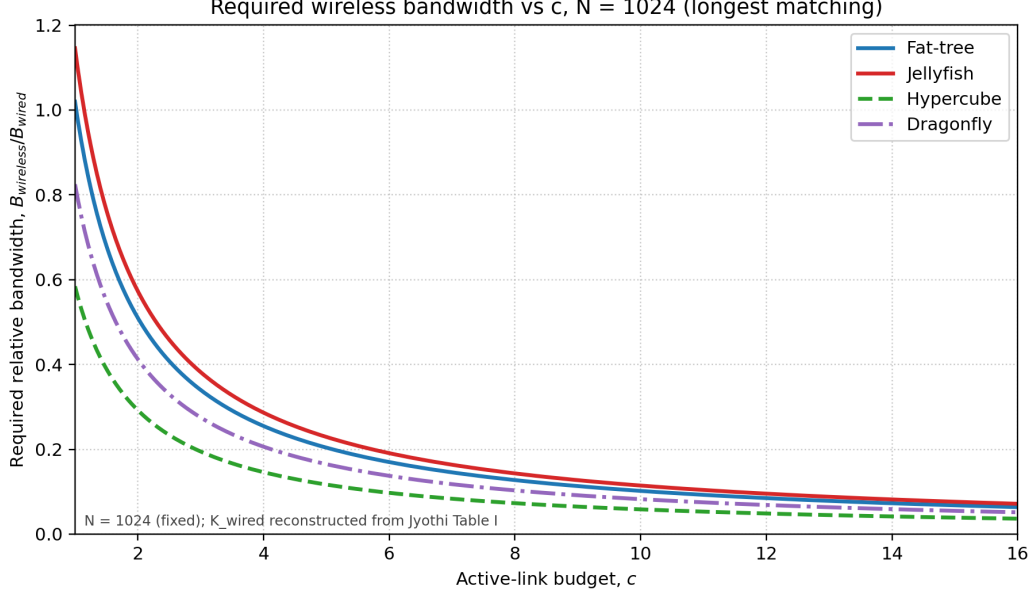


Figure 7: Mode 4 (longest matching): required minimum $B_{\text{wireless}}/B_{\text{wired}}$ vs c at $N = 1024$. The required bandwidth is K_{wired}/c . Hypercube has a notably lower required bandwidth than the other topologies because its longest-matching K_{wired} is low.

8 Discussion

The research question asks what minimum B_{wireless} is required for wireless full-mesh to match wired topology baselines. The answer is not a single topology-independent number. Under ideal bisection, the required relative bandwidth is $BW_{\text{wired}}/BW_{\text{wireless}}$, so Fat-tree and Hypercube require $2/N$, Jellyfish requires $r/((k-r)N)$, Dragonfly follows the group-cut expression, and a d -dimensional Torus requires $8/N^{1+1/d}$. These values are small at large N because the ideal model assumes $\Theta(N^2)$ simultaneous cross-cut wireless links.

The connection-limited model gives the more practical answer. Once each endpoint is restricted to c active peer links, the cross-cut capacity grows as $cN/2$ rather than $N^2/4$. The required bandwidth therefore changes from an N -dominated result to a c -dominated result: Fat-tree and Hypercube require $1/c$, Jellyfish requires $r/(2(k-r)c)$, Dragonfly approaches $1/(2c)$, and the three-dimensional Torus decreases as $4/(cN^{1/3})$. In this sense, c is the wireless analogue of a wired radix or port-count constraint.

The traffic-aware analysis answers the same minimum-bandwidth question from a workload-dependent viewpoint. Instead of comparing only cut capacity, it compares K_{wired} with an analytical K_{wireless} under the same traffic mode. The resulting requirement $B_{\text{wireless}}/B_{\text{wired}} \geq K_{\text{wired}}/K_{\text{wireless}}$ shows why traffic mode matters. Random permutation is the most demanding workload in this comparison: K_{wired} is high (around 1.0–1.4 across topologies) while $K_{\text{wireless}} = c/2$, giving a requirement of roughly $2K_{\text{wired}}/c$. All-to-all has $K_{\text{wireless}} = c/(N-1)$, which makes the wireless denominator small, but K_{wired} is also small for all-to-all because the same high-fanout pattern is difficult for wired topologies as well. This partly offsets the wireless denominator, so the resulting ratio at $N = 1024$ is lower than for random permutation. Longest matching is the most favorable workload because $K_{\text{wireless}} = c$ and K_{wired} is at most comparable to the random-permutation case. Thus, bisection and traffic-aware throughput are complementary answers to the same question:

bisection gives a structural lower requirement, while traffic-aware throughput shows whether the active-link budget can support a concrete traffic pattern.

This workload ordering should not be confused with Jyothi et al.’s traffic-matrix difficulty ordering. Jyothi et al. explicitly state that all-to-all is the easiest traffic matrix for wired topologies, with normalized throughput satisfying $T_{A2A} \geq T_{RM} \geq T_{LM}$, and they characterize longest matching as the near-worst-case representative [6]. In their framework, difficulty is measured by normalized throughput relative to a theoretical lower bound: all-to-all allows topologies to achieve roughly twice their lower bound, while longest matching pushes throughput close to that bound by forcing long routing paths.

That path-length stress mechanism is specific to multi-hop wired topologies. In wireless full-mesh, every communicating pair is one hop apart, so longest matching no longer creates long routing paths; it only activates one peer link per endpoint, giving $K_{\text{wireless}} = c$. The ordering therefore reverses: what is hardest for wired routing (longest matching) becomes most favorable for wireless, and what is easiest for wired routing (all-to-all) still requires a large active-link budget per endpoint. Thus, the difference in workload ordering between this thesis and Jyothi et al. is not an artifact of the $K_{\text{wired}}/K_{\text{wireless}}$ ratio; it reflects the structural property that wireless full-mesh eliminates path-length as a bottleneck entirely.

The practical implication is that wireless full-mesh is plausible only when two conditions hold together. First, the wireless per-link rate must satisfy the required bandwidth ratio for the selected topology and workload. Second, each endpoint must sustain enough simultaneous active links for the target traffic pattern. Increasing only B_{wireless} cannot compensate for a too-small c under all-to-all or other high-fanout traffic, while workloads such as longest matching are much more favorable to the active-link-limited wireless model.

9 Limitations

This thesis does not model a concrete wireless physical layer, antenna design, interference model, MAC protocol, beam scheduling mechanism, or packet-level routing behavior. The wireless K_{wireless} values are analytical required-bandwidth denominators derived from the active-link model. They are not physical wireless simulation outputs.

The active-link budget c is an abstract model parameter. This thesis does not map c to a specific radio architecture, antenna array, beam count, scheduling protocol, or transceiver implementation.

The structural bisection comparison matches endpoint count but does not equalize every hardware resource. It does not fully match switch count, switch radix, cable length, wireless transceiver complexity, power, or deployment cost.

The traffic-aware K_{wired} values in the main result table are reconstructed estimates from Jyothi et al.’s relative-throughput Table I using a local Jellyfish raw- K anchor. They are not exact raw K values reported by Jyothi et al.

The Dragonfly numerical rows are representative of the chosen parameterization. The Dragonfly family has multiple parameter choices, so these rows should not be treated as universal Dragonfly results without a separate parameter study.

Torus is included in the analytical bisection section but not in the current traffic-aware result tables. A traffic-aware Torus comparison would require adding a Torus row to the same throughput workflow or citing a compatible throughput result.

10 Conclusion

This thesis evaluated the per-link wireless bandwidth required for a wireless full-mesh network to match representative wired HPC topology baselines. The analysis separated ideal bisection, connection-limited bisection, and traffic-aware effective throughput so that the meaning of each required-bandwidth expression remains clear.

The main result is that ideal full-mesh bisection gives an optimistic $O(1/N)$ required bandwidth because it assumes $N^2/4$ simultaneous cross-cut wireless links. When each endpoint can maintain only c active wireless links, the structural required bandwidth changes to $O(1/c)$.

The traffic-aware result further shows that the required bandwidth depends on the traffic matrix. At $N = 1024$, random permutation, all-to-all, and longest matching produce different $K_{\text{wired}}/K_{\text{wireless}}$ requirements because they consume the endpoint active-link budget in different ways.

The practical conclusion is that wireless full-mesh feasibility is dominated by active-link capability, not only by per-link bandwidth. A sufficiently high wireless link rate is necessary, but it is not enough unless the system can also provide enough simultaneous active peer connections for the target workload. Future work should connect the abstract c -limited model to concrete wireless hardware, interference-aware scheduling, and packet-level routing experiments.

A Structural Relative-BW Tables at $N = 1024$

The main text uses topology-specific numerical sizes where needed. This appendix fixes $N = 1024$ for a compact structural comparison. The values are relative bisection-width requirements, meaning the minimum $B_{\text{wireless}}/B_{\text{wired}}$ implied by the corresponding bisection model. The Dragonfly row uses the group-level cut expression $(2N + \sqrt{1 + 4N} - 1)/(2N^2)$ for the ideal case and the corresponding connection-limited expression.

Table 9: Ideal full-mesh relative BW values at $N = 1024$.

Topology	Relative $BW_{\text{wired}}/BW_{\text{wireless}}$
Fat-tree	0.001953
Jellyfish, $r/(k-r) = 4$	0.003906
Hypercube	0.001953
Dragonfly, group-level cut expression	0.001007
Torus, $d = 3$	0.000775

Table 10: Connection-limited relative BW values at $N = 1024$.

Topology	$c = 1$	$c = 2$	$c = 3$	$c = 4$
Fat-tree	1.000000	0.500000	0.333333	0.250000
Jellyfish, $r/(k-r) = 4$	2.000000	1.000000	0.666667	0.500000
Hypercube	1.000000	0.500000	0.333333	0.250000
Dragonfly, group-level cut expression	0.515383	0.257692	0.171794	0.128846
Torus, $d = 3$	0.396850	0.198425	0.132283	0.099213

B Local Raw K and Jyothi Table I Consistency

Jyothi et al. report relative throughput after normalizing each topology by a same-equipment random graph, so Table I gives comparison ratios rather than raw LP objective values [6]. The following table keeps the local Fat-tree comparison used to decide whether the local Jellyfish raw K values are a feasible anchor for reconstructing approximate raw K values from Table I. The agreement is not exact, so this thesis uses the reconstruction only for trend comparison.

Table 11: Local Fat-tree relative throughput compared with Jyothi et al. Table I.

Workload	Mode	Local raw K	Local rel.	Jyothi rel.	Reconstructed K
Random perm.	0	1.00000090	0.717630	0.73	1.00904368
All-to-all	1	0.00098424	0.641169	0.65	0.00099887
Longest match.	4	0.99999985	0.874961	0.89	1.01958967

C Local TopoBench K Sensitivity Table

The main traffic-aware table uses reconstructed estimates from Jyothi et al. Table I rather than direct local K values. The following table preserves the direct local values as a sensitivity check only. Each cell substitutes $c = 1, 2, 3, 4$ into the same mode-specific expressions used in Table 8: $2K_{\text{wired}}/c$ for random permutation, $K_{\text{wired}}(N-1)/c$ for all-to-all, and K_{wired}/c for longest matching. These values are not used as the main paper-aligned result table.

Table 12: Direct local traffic-aware required bandwidth values after substituting $c = 1, 2, 3, 4$.

Topology	Workload	$c = 1$	$c = 2$	$c = 3$	$c = 4$
Fat-tree	Random permutation	2.000000	1.000000	0.666667	0.500000
Fat-tree	All-to-all	1.006632	0.503316	0.335544	0.251658
Fat-tree	Longest matching	1.000000	0.500000	0.333333	0.250000
Jellyfish	Random permutation	2.779300	1.389650	0.926433	0.694825
Jellyfish	All-to-all	1.571328	0.785664	0.523776	0.392832
Jellyfish	Longest matching	1.145690	0.572845	0.381897	0.286423
Hypercube	Random permutation	3.782060	1.891030	1.260687	0.945515
Hypercube	All-to-all	1.997919	0.998960	0.665973	0.499480
Hypercube	Longest matching	1.000000	0.500000	0.333333	0.250000
Dragonfly	Random permutation	1.592964	0.796482	0.530988	0.398241
Dragonfly	All-to-all	1.020185	0.510092	0.340062	0.255046
Dragonfly	Longest matching	0.523901	0.261950	0.174634	0.130975

References

- [1] Al-Fares, Mohammad, Alexander Loukissas, and Amin Vahdat. “A Scalable, Commodity Data Center Network Architecture.” *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, 2008, pp. 63–74.
- [2] Singla, Ankit, et al. “Jellyfish: Networking Data Centers Randomly.” *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, 2012.

- [3] Kim, John, et al. “Technology-Driven, Highly-Scalable Dragonfly Topology.” *ACM SIGARCH Computer Architecture News*, vol. 36, no. 3, 2008, pp. 77–88.
- [4] Arjona Aroca, Jordi, and Antonio Fernández Anta. “Bisection (Band)Width of Product Networks with Application to Data Centers.” *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, 2014, pp. 570–580. DOI: 10.1109/TPDS.2013.95.
- [5] Leighton, F. Thomson. *Introduction to Parallel Algorithms and Architectures: Arrays, Trees, Hypercubes*. Morgan Kaufmann, 1992.
- [6] Jyothi, Sangeetha Abdu, et al. “Measuring and Understanding Throughput of Network Topologies.” *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, IEEE, 2016, pp. 761–772. DOI: 10.1109/SC.2016.64.